

ForgeSight

AI-Assisted Visual Inspection and Defect Triage Platform

Document type: Product and technical specification

Project type: Public portfolio project

Primary domain: Industrial computer vision

Primary objective: Demonstrate end-to-end AI, backend, frontend, database, testing, deployment and operational engineering capability

1. Executive summary

ForgeSight is a web-based visual inspection platform that analyses images of manufactured components, identifies potential defects, highlights the affected regions and presents the results to a human reviewer.

Users can:

1. Create an inspection.
2. Upload one or more component images.
3. Run a selected model version.
4. Review defect classifications, confidence scores and segmentation overlays.
5. Accept, reject or amend the model's findings.
6. Compare model versions and performance.
7. Export inspection results and audit records.

The platform uses:

- PyTorch for model development.
- OpenCV for image processing.
- FastAPI for the backend API.
- PostgreSQL for structured data.
- React and TypeScript for the user interface.
- MLflow for experiment and model tracking.
- Docker Compose for local deployment.
- ONNX Runtime as an optional optimised inference path.
- Automated tests and CI pipelines.

The system should be implemented as a **modular monolith with a separate background worker**, rather than a collection of unnecessary microservices.

2. Portfolio purpose

The project should prove that the developer can do more than train a model in a notebook.

It should demonstrate:

- Problem formulation.
- Data preparation.
- Classical image processing.
- Deep-learning model development.
- Experiment tracking.
- Model evaluation.
- Inference service construction.
- Asynchronous job processing.
- REST API design.
- Database modelling.
- Frontend application development.
- Human-in-the-loop workflows.
- Authentication and authorisation.
- Testing across multiple layers.
- Containerisation.
- CI/CD.
- Technical documentation.
- Security and operational thinking.

The final project should be explainable as:

A production-style computer-vision system that takes a model from dataset exploration through deployment, review and monitoring.

3. Project goals

3.1 Primary goals

The system must:

- Detect whether an uploaded component image is defective.
- Produce a probability or confidence score.
- Localise the suspected defect using a segmentation mask or anomaly heatmap.
- Store every inference result with its model version.
- Present results through a usable web interface.
- Allow a reviewer to confirm or reject the result.
- Maintain a complete review and inference audit trail.
- Track model experiments and evaluation metrics.
- Run locally through a documented one-command setup.
- Pass automated tests through CI.

3.2 Secondary goals

The system should:

- Support batch uploads.

- Support multiple model versions.
- Compare model performance.
- Export results.
- Process inference asynchronously.
- Produce meaningful logs and health information.
- Allow rejected predictions to become future training candidates.
- Support deployment to a public cloud environment.

3.3 Non-goals

The first release will not attempt to provide:

- Safety certification.
 - Real-time production-line control.
 - Live autonomous decision-making.
 - Classified or export-controlled data processing.
 - Facial recognition.
 - Surveillance of people.
 - Weapon detection or targeting.
 - Full dataset-labelling functionality.
 - Multi-tenant billing.
 - Large-scale Kubernetes deployment.
 - An unnecessarily distributed microservice architecture.
-

4. Target users

4.1 Inspector

An inspector performs or reviews visual inspections.

Responsibilities:

- Create inspections.
- Upload images.
- Review model findings.
- Mark findings as correct or incorrect.
- Add notes.
- Export inspection results.

4.2 Quality engineer

A quality engineer monitors defect patterns and model behaviour.

Responsibilities:

- Review aggregate defect statistics.
- Search prior inspections.
- Compare model versions.

- Identify false-positive and false-negative trends.
- Export data for analysis.

4.3 ML engineer

An ML engineer trains, evaluates and promotes models.

Responsibilities:

- Run training experiments.
- Review metrics in MLflow.
- Register model artefacts.
- Promote approved model versions.
- Monitor inference performance.
- Investigate data or model drift.

4.4 Administrator

An administrator manages application access and configuration.

Responsibilities:

- Manage users and roles.
 - Activate or deactivate model versions.
 - Review audit logs.
 - Configure system limits.
-

5. Core user journeys

5.1 New inspection

1. User signs in.
2. User selects **New inspection**.
3. User enters:
 4. Component category.
 5. Part or batch identifier.
 6. Inspection notes.
 7. User uploads one or more images.
 8. The application validates and stores the images.
 9. The user selects an active model version or accepts the default.
10. The system creates an inference job.
11. The frontend displays job progress.
12. Results are displayed when processing finishes.
13. The user reviews and confirms the findings.

5.2 Defect review

1. Reviewer opens the review queue.
2. Reviewer selects an unreviewed result.
3. The interface displays:
4. Original image.
5. Defect overlay.
6. Predicted class.
7. Confidence.
8. Model version.
9. Processing metadata.
10. Reviewer chooses:
11. Accept.
12. Reject.
13. Amend classification.
14. Mark uncertain.
15. Reviewer optionally enters a note.
16. The decision is persisted in the audit trail.

5.3 Model comparison

1. ML engineer opens the model page.
2. The application displays registered model versions.
3. User compares:
4. Validation metrics.
5. Production review metrics.
6. Inference latency.
7. Date registered.
8. Current lifecycle status.
9. User selects a model as the active production model.
10. The change is recorded in the audit log.

5.4 Feedback dataset creation

1. ML engineer filters reviewed predictions.
2. Engineer selects rejected or amended samples.
3. System creates an export manifest.
4. Manifest contains:
5. Image identifier.

6. Original prediction.
 7. Reviewer correction.
 8. Model version.
 9. Confidence.
 10. Review notes.
 11. Export can be used in a later retraining cycle.
-

6. Scope prioritisation

6.1 Must have

- Authentication.
- Role-based access control.
- Inspection creation.
- Image upload.
- File validation.
- Image storage.
- PyTorch inference.
- Defect classification.
- Segmentation mask or anomaly heatmap.
- Persistent inference results.
- Human review workflow.
- Model-version tracking.
- MLflow experiment tracking.
- PostgreSQL database.
- React interface.
- Docker Compose setup.
- Automated tests.
- CI pipeline.
- Technical documentation.

6.2 Should have

- Batch upload.
- Asynchronous job processing.
- Search and filtering.
- Result export.
- Model comparison page.
- Reviewer notes.
- Audit logs.
- ONNX inference.
- Structured application logs.
- Health and readiness endpoints.

6.3 Could have

- Video upload and frame extraction.
- Webcam capture.

- Active-learning sample selection.
- Annotation editing.
- Model explainability views.
- Prometheus metrics.
- Grafana dashboard.
- Object-storage integration.
- Cloud deployment.
- GPU-backed inference.
- Email or webhook notifications.
- Dark mode.

6.4 Will not have in the first release

- Kubernetes.
 - Service mesh.
 - Kafka.
 - Multi-region deployment.
 - Complex enterprise identity federation.
 - Multiple independently deployed business microservices.
 - Automated model promotion without human approval.
-

7. Dataset and machine-learning problem

7.1 Recommended dataset

Use the **MVTec Anomaly Detection dataset** or another public industrial-defect dataset with image-level and pixel-level labels.

The dataset should contain:

- Normal component images.
- Defective component images.
- Defect categories.
- Pixel-level masks where available.
- Multiple component types.

No proprietary workplace data should be used in the public repository.

7.2 ML tasks

The project should support two related tasks.

Task A: defect classification

Input:

- Component image.

Output:

- Normal or defective.
- Optional defect category.
- Probability score.

Task B: defect localisation

Input:

- Component image.

Output:

- Pixel-level segmentation mask or anomaly heatmap.
- Defect region bounding box derived from the mask.
- Defect area ratio.
- Severity estimate.

7.3 Model stages

Baseline model

Use a pretrained ResNet18 or EfficientNet classifier.

Purpose:

- Establish a reproducible baseline.
- Test the complete data and deployment pipeline.
- Produce initial classification metrics.

Main supervised model

Use one of:

- U-Net.
- DeepLabV3.
- Segmentation head attached to a pretrained encoder.

Purpose:

- Produce pixel-level defect masks.
- Demonstrate segmentation.
- Support visual overlays.

Advanced model

Implement an anomaly-detection approach such as:

- Autoencoder.
- PatchCore.
- PaDiM.

- Teacher-student anomaly detection.

Purpose:

- Demonstrate learning where defective examples are limited.
 - Cover methods beyond conventional supervised learning.
 - Compare supervised and anomaly-detection approaches.
-

8. ML pipeline specification

8.1 Data ingestion

The training pipeline must:

- Read a documented dataset directory structure.
- Validate file formats.
- Verify masks match image dimensions.
- Detect corrupt images.
- Produce dataset statistics.
- Generate a deterministic train, validation and test split.
- Prevent the same physical object or image variant appearing across splits.
- Record a dataset version or hash.

8.2 Preprocessing

OpenCV should be used for:

- Image decoding.
- Colour-space conversion.
- Resizing.
- Aspect-ratio preservation.
- Padding.
- Contrast inspection.
- Optional histogram equalisation or CLAHE.
- Blur and sharpness measurements.
- Mask post-processing.
- Bounding-box extraction.

Preprocessing must be shared between training and inference wherever possible.

8.3 Augmentation

Use Albumentations for controlled training augmentation.

Potential augmentations:

- Horizontal or vertical flips where physically valid.
- Small rotations.

- Random crops.
- Brightness and contrast changes.
- Gaussian noise.
- Mild blur.
- Perspective changes within realistic limits.

Augmentations must not create physically implausible defects or destroy label validity.

8.4 Training configuration

Training settings should be controlled through versioned configuration files.

Configuration should include:

- Dataset path.
- Model architecture.
- Input dimensions.
- Batch size.
- Learning rate.
- Optimiser.
- Weight decay.
- Number of epochs.
- Scheduler.
- Augmentation settings.
- Random seed.
- Classification threshold.
- Segmentation threshold.

Recommended format:

```
experiment_name: unet_resnet34_v1
seed: 42

data:
  image_size: [256, 256]
  batch_size: 16
  num_workers: 4

model:
  architecture: unet
  encoder: resnet34
  pretrained: true

training:
  epochs: 50
  optimiser: adamw
  learning_rate: 0.0003
  weight_decay: 0.0001
```

```
scheduler: cosine
early_stopping_patience: 8
```

8.5 Training requirements

The training code must support:

- CPU and GPU execution.
- Reproducible random seeds.
- Training and validation loops.
- Checkpointing.
- Early stopping.
- Learning-rate scheduling.
- Mixed precision where supported.
- Resume from checkpoint.
- Metric logging.
- Configuration capture.
- Model artefact export.

8.6 MLflow tracking

Each experiment should record:

- Git commit.
- Dataset version.
- Model architecture.
- Hyperparameters.
- Training and validation metrics.
- Confusion matrix.
- Precision-recall curve.
- Example predictions.
- Final model weights.
- ONNX model where available.
- Training duration.
- Hardware information.
- Package environment.

8.7 Evaluation metrics

Classification metrics

- Accuracy.
- Balanced accuracy.
- Precision.
- Recall.
- F1 score.
- AUROC.
- Precision-recall AUC.
- False-negative rate.
- Calibration error.

- Confusion matrix.

False-negative rate should be treated as particularly important because missed defects may be more costly than unnecessary manual reviews.

Segmentation metrics

- Intersection over Union.
- Dice coefficient.
- Pixel precision.
- Pixel recall.
- Image-level anomaly score.
- Boundary quality where practical.

Operational metrics

- Mean inference latency.
- P95 inference latency.
- Memory usage.
- Model size.
- Throughput.
- Failure rate.

8.8 Threshold selection

Classification and segmentation thresholds must be selected using validation data rather than arbitrary defaults.

Threshold selection should consider:

- False-negative cost.
- False-positive review burden.
- Precision-recall trade-off.
- Calibration.
- Different component categories.

8.9 Model packaging

The final model package must include:

- Model weights.
- Architecture configuration.
- Class labels.
- Preprocessing configuration.
- Input and output schema.
- Thresholds.
- Model version.
- Model card.
- Example input and output.

8.10 Model card

Each promoted model must have a model card containing:

- Intended use.
 - Training dataset.
 - Evaluation dataset.
 - Metrics.
 - Known limitations.
 - Failure modes.
 - Supported image types.
 - Input requirements.
 - Ethical considerations.
 - Deployment version.
 - Date produced.
-

9. System architecture

9.1 Architectural style

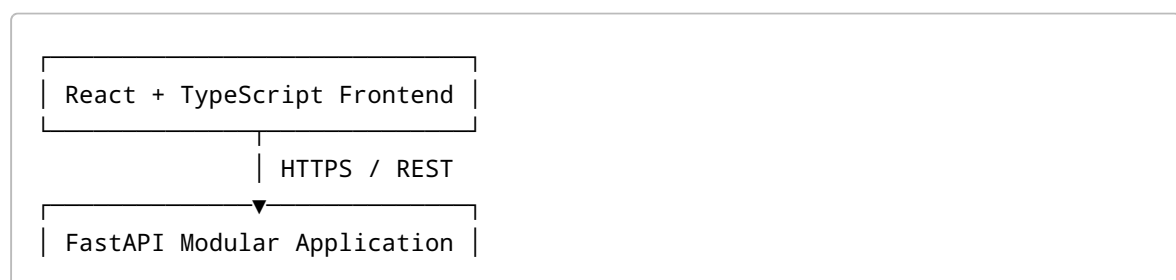
Use a modular monolith with the following runtime components:

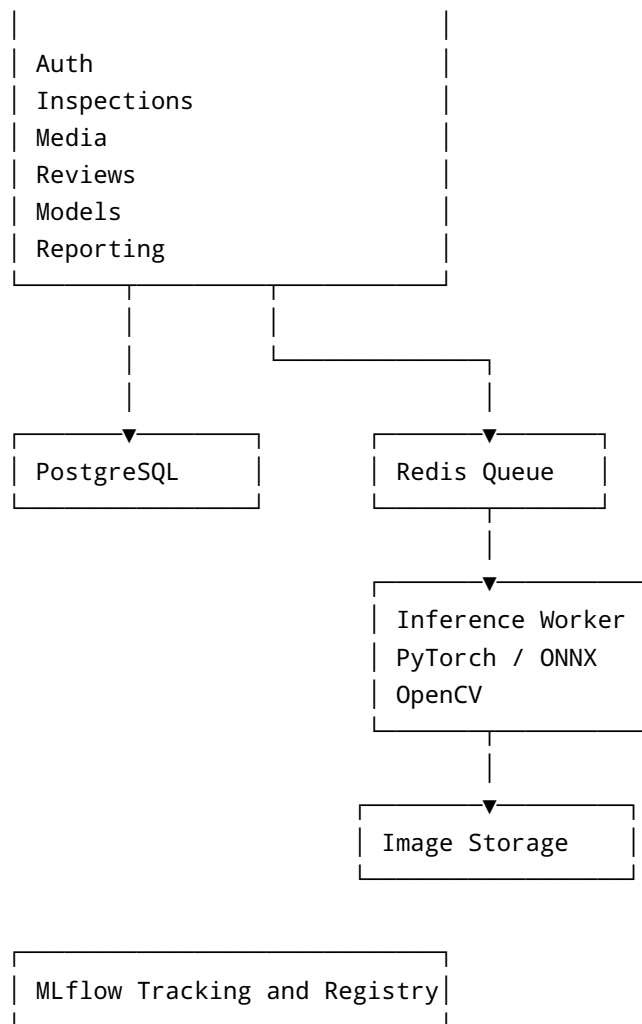
1. Web frontend.
2. FastAPI application.
3. Background inference worker.
4. PostgreSQL database.
5. Redis queue.
6. MLflow tracking server.
7. Image or object storage.

The API and worker may share a Python package containing:

- Domain models.
- Database access.
- Inference code.
- Validation.
- Configuration.
- Logging utilities.

9.2 Logical architecture





9.3 Repository structure

```

forgesight/
├── apps/
│   ├── web/
│   │   ├── src/
│   │   ├── tests/
│   │   └── package.json
│   ├── api/
│   │   ├── forgesight_api/
│   │   ├── tests/
│   │   └── pyproject.toml
│   └── worker/
│       ├── forgesight_worker/
│       └── tests/
├── packages/
│   ├── python_common/
│   └── api_client/
└── ml/
  
```

```
|   |─ configs/
|   |─ datasets/
|   |─ training/
|   |─ evaluation/
|   |─ inference/
|   |─ notebooks/
|   └─ tests/
|─ infra/
|   |─ docker/
|   |─ compose/
|   └─ cloud/
|─ migrations/
|─ scripts/
|─ docs/
|   |─ architecture/
|   |─ adr/
|   |─ api/
|   |─ ml/
|   └─ security/
|─ .github/
|   └─ workflows/
|─ docker-compose.yml
|─ README.md
└─ Makefile
```

10. Technology stack

10.1 Frontend

- React.
- TypeScript.
- Vite.
- React Router.
- TanStack Query.
- React Hook Form.
- Zod.
- CSS Modules, Tailwind or another consistent styling method.
- Vitest.
- React Testing Library.
- Playwright.

10.2 Backend

- Python.
- FastAPI.
- Pydantic.
- SQLAlchemy.

- Alembic.
- PostgreSQL.
- Redis.
- Celery, Dramatiq or RQ.
- Pytest.
- HTTPX.
- Structured logging.

10.3 Machine learning

- PyTorch.
- Torchvision.
- OpenCV.
- NumPy.
- pandas.
- scikit-learn.
- Albumentations.
- MLflow.
- ONNX.
- ONNX Runtime.

10.4 Infrastructure

- Docker.
- Docker Compose.
- GitHub Actions or GitLab CI.
- Managed PostgreSQL for cloud deployment.
- Blob or object storage.
- Container registry.
- Optional Azure Container Apps or AWS ECS.

10.5 Quality and security tooling

Python:

- Ruff.
- Black if formatting is not delegated to Ruff.
- MyPy or Pyright.
- Bandit.
- pip-audit.

TypeScript:

- ESLint.
- Prettier.
- TypeScript strict mode.
- npm audit or equivalent dependency scanning.

Containers:

- Trivy.

- Multi-stage Docker builds.
 - Non-root runtime users.
-

11. Backend modules

11.1 Authentication module

Responsibilities:

- User login.
- Logout.
- Password hashing.
- Session or token validation.
- Role enforcement.
- Current-user retrieval.

Recommended security model:

- Argon2 password hashing.
- Short-lived access token.
- Secure HTTP-only cookie.
- SameSite cookie policy.
- CSRF protection where required.
- Role-based access control.

Roles:

- Inspector.
- Quality engineer.
- ML engineer.
- Administrator.

11.2 Inspection module

Responsibilities:

- Create inspection.
- Update inspection metadata.
- List inspections.
- Filter inspections.
- Retrieve inspection details.
- Close or archive inspections.

11.3 Media module

Responsibilities:

- Validate uploaded files.

- Store original images.
- Generate thumbnails.
- Record dimensions and hashes.
- Prevent duplicate accidental uploads.
- Provide controlled media access.

11.4 Inference module

Responsibilities:

- Create inference jobs.
- Select model version.
- Queue work.
- Track job status.
- Store results.
- Handle retries.
- Reject incompatible inputs.

11.5 Review module

Responsibilities:

- Display pending reviews.
- Record reviewer decision.
- Store corrected labels.
- Store reviewer notes.
- Prevent accidental overwrites.
- Maintain decision history.

11.6 Model-management module

Responsibilities:

- List model versions.
- Record model metadata.
- Activate or deactivate models.
- Mark lifecycle state.
- Select default model.
- Link to MLflow run.
- Prevent deletion of models used by historical inspections.

11.7 Reporting module

Responsibilities:

- Defect counts.
- Defect rates.
- Review agreement.
- False-positive rate.
- False-negative observations.

- Model-version comparisons.
- CSV or JSON exports.

11.8 Audit module

Responsibilities:

- Record security-relevant and business-relevant actions.
 - Store actor, action, resource and timestamp.
 - Preserve previous and new values for important changes.
 - Support administrative search.
-

12. Database design

12.1 Core entities

users

- id
- email
- password_hash
- display_name
- role
- is_active
- created_at
- updated_at
- last_login_at

inspections

- id
- component_type
- part_identifier
- batch_identifier
- status
- notes
- created_by
- created_at
- updated_at
- completed_at

media_files

- id
- inspection_id
- storage_key
- original_filename
- content_type
- file_size

- width
- height
- sha256
- created_at

model_versions

- id
- name
- semantic_version
- model_type
- framework
- storage_uri
- mlflow_run_id
- preprocessing_config
- threshold_config
- lifecycle_status
- is_default
- created_at
- activated_at

inference_jobs

- id
- inspection_id
- model_version_id
- status
- requested_by
- queued_at
- started_at
- completed_at
- error_code
- error_message

inference_results

- id
- job_id
- media_file_id
- predicted_label
- confidence
- anomaly_score
- defect_area_ratio
- segmentation_storage_key
- overlay_storage_key
- bounding_boxes
- latency_ms
- created_at

reviews

- id

- inference_result_id
- reviewer_id
- decision
- corrected_label
- notes
- created_at
- updated_at

audit_events

- id
- actor_id
- action
- resource_type
- resource_id
- previous_value
- new_value
- created_at
- request_id

12.2 Important database constraints

- User emails must be unique.
- A model semantic version must be unique.
- Every inference result must reference an immutable model version.
- A review must reference an existing result.
- Historical model versions must not be deleted if referenced.
- File hash and inspection identifier may be used to warn about duplicates.
- Lifecycle status values must be constrained.
- Foreign keys must be indexed where appropriate.

12.3 Model lifecycle states

- Development.
- Candidate.
- Approved.
- Active.
- Retired.
- Rejected.

Only one model per supported task should be active by default.

13. API specification

Base path:

/api/v1

13.1 Authentication

POST /auth/login

Request:

```
{
  "email": "reviewer@example.com",
  "password": "example-password"
}
```

Response:

```
{
  "user": {
    "id": "uuid",
    "email": "reviewer@example.com",
    "displayName": "Reviewer",
    "role": "inspector"
  }
}
```

POST /auth/logout

Terminates the current session.

GET /auth/me

Returns the current authenticated user.

13.2 Inspections

POST /inspections

Creates an inspection.

GET /inspections

Supports:

- Pagination.
- Status filtering.
- Component-type filtering.
- Date filtering.
- Free-text search.

GET `/inspections/{inspection_id}`

Returns inspection metadata, media, jobs and review summary.

PATCH `/inspections/{inspection_id}`

Updates editable metadata.

13.3 Media

POST `/inspections/{inspection_id}/media`

Accepts multipart image upload.

Validation:

- Allowed MIME type.
- Allowed extension.
- Maximum size.
- Successful image decode.
- Width and height limits.
- No embedded executable content.
- Generated server-side storage name.

GET `/media/{media_id}`

Returns authorised access to an image or generated overlay.

13.4 Inference

POST `/inspections/{inspection_id}/inference-jobs`

Request:

```
{
  "modelVersionId": "uuid",
  "mediaIds": ["uuid", "uuid"]
}
```

Response:

```
{
  "jobId": "uuid",
  "status": "queued"
}
```

GET /inference-jobs/{job_id}

Response:

```
{
  "id": "uuid",
  "status": "completed",
  "progress": {
    "completed": 2,
    "total": 2
  },
  "results": ["uuid", "uuid"]
}
```

GET /inference-results/{result_id}

Returns:

- Prediction.
- Confidence.
- Mask or heatmap location.
- Bounding boxes.
- Model version.
- Latency.
- Review state.

13.5 Reviews

GET /reviews/queue

Returns unreviewed or uncertain results.

POST /inference-results/{result_id}/review

Request:

```
{
  "decision": "amended",
  "correctedLabel": "scratch",
  "notes": "Model identified the defect but assigned the wrong category."
}
```

13.6 Models

GET /models

Lists model versions and lifecycle state.

GET `/models/{model_id}`

Returns model metadata and evaluation summary.

POST `/models/{model_id}/activate`

Administrator or ML-engineer only.

POST `/models/{model_id}/retire`

Removes the model from new use while preserving historical references.

13.7 Reporting

GET `/reports/summary`

Returns aggregate inspection metrics.

GET `/reports/model-comparison`

Returns operational metrics grouped by model version.

GET `/exports/review-feedback`

Returns reviewed data as a manifest.

13.8 Operations

GET `/health/live`

Confirms the process is running.

GET `/health/ready`

Confirms required dependencies are available.

GET `/metrics`

Optional Prometheus-compatible metrics endpoint.

14. Background inference processing

14.1 Job lifecycle

```
graph TD
    A[queued] --> B[running]
```

↓
completed

Failure paths:

queued → failed
running → retrying → running
running → failed
running → cancelled

14.2 Worker behaviour

The worker must:

1. Receive a job identifier.
2. Retrieve job and model metadata.
3. Mark the job as running.
4. Load the requested model.
5. Retrieve the image.
6. Validate dimensions and format.
7. Apply preprocessing.
8. Run inference.
9. Apply post-processing.
10. Store masks and overlays.
11. Save structured results.
12. Mark progress.
13. Mark job complete or failed.

14.3 Idempotency

A retried task must not create duplicate results.

Possible approach:

- Use a unique constraint on job ID and media-file ID.
- Use transactional result writes.
- Check whether a completed result already exists before processing.

14.4 Model loading

The worker should:

- Load models once where practical.
- Cache active model instances by version.
- Validate model metadata at startup.
- Reject unsupported model formats.
- Record model load failures.
- Avoid loading user-supplied executable model artefacts.

15. Frontend specification

15.1 Main pages

Login

- Email.
- Password.
- Error feedback.
- Keyboard-accessible form.
- No sensitive error details.

Dashboard

Display:

- Recent inspections.
- Pending reviews.
- Defect-rate summary.
- Active model.
- Recent failures.
- Model-review agreement.

New inspection

Include:

- Metadata form.
- Drag-and-drop upload.
- Upload preview.
- File validation errors.
- Model selection.
- Submit and run controls.

Inspection detail

Display:

- Inspection metadata.
- Uploaded images.
- Job state.
- Results.
- Review state.
- Audit timeline.

Result detail

Display:

- Original image.
- Overlay viewer.
- Opacity slider.
- Zoom and pan.
- Predicted label.
- Confidence.
- Defect-area ratio.
- Bounding boxes.
- Model version.
- Review controls.

Review queue

Support:

- Next and previous review.
- Keyboard shortcuts.
- Filter by model.
- Filter by confidence.
- Filter by component type.
- Accept, reject, amend and uncertain decisions.

Models

Display:

- Version.
- Type.
- Lifecycle state.
- Validation metrics.
- Operational metrics.
- Activation history.
- Link to model card.

Reports

Display:

- Inspection volume.
- Defect categories.
- Review outcomes.
- Model agreement.
- Latency.
- Date and model filters.

15.2 Image overlay viewer

The image viewer should support:

- Original image.
- Segmentation mask.
- Heatmap.
- Bounding box.
- Overlay opacity control.
- Fit-to-screen.
- Native resolution.
- Pixel coordinate display.
- Downloadable comparison image.

15.3 Frontend state

Use:

- TanStack Query for server state.
- Local React state for view controls.
- URL query parameters for filters.
- React Hook Form for forms.
- Zod for client-side validation.

Avoid introducing a global state framework unless the application genuinely requires it.

15.4 Accessibility

The frontend must:

- Use semantic HTML.
- Support keyboard navigation.
- Provide visible focus.
- Provide text equivalents for status indicators.
- Avoid colour-only defect communication.
- Use sufficient contrast.
- Label all form controls.
- Announce asynchronous job completion where practical.

16. Security requirements

16.1 Authentication and authorisation

- Passwords must be hashed with a modern password-hashing algorithm.
- Protected routes must require authentication.
- Server-side role checks must protect privileged actions.
- Frontend route hiding must not be treated as authorisation.
- Sessions must expire.

- Cookies must be secure in deployed environments.
- Login attempts should be rate limited.

16.2 File-upload security

- Validate actual file content, not only extension.
- Restrict MIME types.
- Set maximum file size.
- Reject images that cannot be decoded.
- Generate server-controlled filenames.
- Store files outside application code directories.
- Do not execute uploaded content.
- Strip unnecessary metadata where appropriate.
- Limit decompressed image dimensions to prevent image bombs.

16.3 API security

- Validate all request bodies.
- Use parameterised database access.
- Restrict CORS.
- Apply request-size limits.
- Avoid sensitive values in logs.
- Return generic internal-error responses.
- Add request identifiers.
- Apply rate limiting to expensive inference endpoints.

16.4 Secrets

Secrets must not be committed to the repository.

Use environment variables or a secret manager for:

- Database credentials.
- Session keys.
- Object-storage credentials.
- MLflow credentials.
- Cloud credentials.

Provide a safe `.env.example`.

16.5 Dependency and supply-chain security

CI should include:

- Python dependency audit.
- Node dependency audit.
- Static security analysis.
- Container image scan.
- Lockfiles.
- Pinned major dependency versions.

- Automated dependency-update configuration.

16.6 Audit requirements

The platform should audit:

- Login success and failure.
 - Inspection creation.
 - Inference submission.
 - Review decisions.
 - Model activation.
 - Model retirement.
 - Role changes.
 - Export generation.
-

17. Reliability and performance requirements

17.1 Performance targets

For a portfolio deployment:

- Non-inference API requests should generally complete within 300 ms at P95 under light load.
- Image uploads should provide progress feedback.
- Inference jobs should begin without blocking the API process.
- Single-image GPU inference should target less than two seconds.
- CPU inference should remain practically demonstrable.
- The frontend should remain responsive during processing.

These are engineering targets, not safety guarantees.

17.2 Reliability

The system must:

- Preserve historical inference results.
- Preserve model-version references.
- Retry transient worker failures.
- Avoid duplicate result creation.
- Recover cleanly after container restart.
- Use database migrations.
- Validate configuration at startup.
- Provide health endpoints.

17.3 Data integrity

- Important writes should use transactions.
- Model activation must be atomic.
- An inspection result must never silently change model version.

- Review history should be append-only or versioned.
 - Generated outputs should be linked to immutable source images.
-

18. Observability

18.1 Logging

Use structured JSON logs in deployed environments.

Each request log should include:

- Timestamp.
- Severity.
- Service.
- Request ID.
- User ID where appropriate.
- Route.
- Status code.
- Duration.
- Error category.

Inference logs should include:

- Job ID.
- Media ID.
- Model version.
- Preprocessing duration.
- Inference duration.
- Post-processing duration.
- Result status.

Do not log:

- Passwords.
- Session tokens.
- Raw secret values.
- Full image content.

18.2 Metrics

Recommended application metrics:

- Request count.
- Request latency.
- Error count.
- Job queue depth.
- Job duration.
- Inference duration.
- Inference failure count.

- Review queue depth.
- Prediction distribution.
- Confidence distribution.
- Model-version usage.

18.3 Tracing

Optional OpenTelemetry tracing can connect:

- Frontend request.
 - API operation.
 - Queue submission.
 - Worker execution.
 - Database writes.
-

19. Testing strategy

19.1 ML unit tests

Test:

- Dataset loading.
- Invalid file handling.
- Preprocessing output dimensions.
- Normalisation.
- Mask alignment.
- Model input and output shape.
- Thresholding.
- Bounding-box extraction.
- ONNX and PyTorch output agreement.
- Deterministic behaviour on fixed fixtures.

19.2 Model regression tests

Maintain a small, version-controlled test fixture set.

Verify:

- Predictions remain within expected ranges.
- No unexpected class-label changes.
- Latency remains within a broad limit.
- Exported model produces valid results.
- Preprocessing configuration matches training.

Do not store the entire training dataset in the Git repository.

19.3 Backend unit tests

Test:

- Authentication logic.
- Role checks.
- Inspection creation.
- File validation.
- Job creation.
- Model selection.
- Review decisions.
- Export generation.
- Audit events.
- Error mapping.

19.4 Database integration tests

Test:

- Migrations.
- Constraints.
- Transaction rollback.
- Model activation atomicity.
- Duplicate job handling.
- Historical references.
- Pagination and filtering.

Use a real PostgreSQL test container where practical.

19.5 API contract tests

Test:

- Request schema.
- Response schema.
- Authentication.
- Authorisation.
- Status codes.
- Error bodies.
- Pagination metadata.
- Idempotency.

19.6 Frontend tests

Unit and component tests

- Forms.
- Validation.
- Loading states.
- Error states.

- Role-based controls.
- Overlay controls.
- Result formatting.

End-to-end tests

Use Playwright to test:

1. Login.
2. Create inspection.
3. Upload image.
4. Submit inference.
5. Wait for completion.
6. View result.
7. Submit review.
8. Confirm review persistence.
9. View model metadata.
10. Log out.

19.7 Security tests

Test:

- Unauthenticated access.
- Incorrect-role access.
- Malicious filenames.
- Unsupported media.
- Oversized files.
- Invalid tokens.
- Cross-user resource access.
- Rate limits.
- Missing CSRF protection where relevant.

19.8 Performance tests

Use a lightweight load-testing tool to test:

- Inspection listing.
- Result retrieval.
- Concurrent job submission.
- Batch upload.
- Queue behaviour.

20. Continuous integration

The CI pipeline should run on every pull request.

20.1 Backend jobs

- Install dependencies.
- Lint.
- Format check.
- Type check.
- Unit tests.
- Integration tests.
- Security scan.
- Coverage report.

20.2 Frontend jobs

- Install locked dependencies.
- Lint.
- Type check.
- Unit tests.
- Production build.
- End-to-end tests.

20.3 ML jobs

Run only lightweight checks on every pull request:

- Dataset-loader tests.
- Model-shape tests.
- Small fixture inference.
- ONNX export test.
- Configuration validation.

Full model training should not run on every commit.

20.4 Container jobs

- Build images.
- Scan images.
- Start Docker Compose.
- Run smoke tests.
- Publish images only from approved branches or releases.

20.5 Release pipeline

On a tagged release:

- Build versioned images.
- Generate release notes.
- Publish container images.
- Apply database migrations through a controlled step.
- Deploy application.
- Run post-deployment smoke tests.

21. Local development

21.1 Required commands

Provide a Makefile or equivalent task runner.

Expected commands:

```
make setup
make dev
make test
make lint
make typecheck
make train-baseline
make export-model
make compose-up
make compose-down
make migrate
make seed
```

21.2 One-command demonstration

The repository should support:

```
docker compose up --build
```

This should start:

- Frontend.
- API.
- Worker.
- PostgreSQL.
- Redis.
- MLflow.
- Local object storage if used.

A seeded demonstration account and example images may be included.

21.3 Developer documentation

Document:

- Prerequisites.
- Environment variables.
- Database setup.

- Model placement.
 - Dataset setup.
 - Training commands.
 - Testing commands.
 - Common failure modes.
 - Architecture overview.
-

22. Cloud deployment

22.1 Recommended initial target

For an Azure-oriented deployment:

- Azure Container Registry.
- Azure Container Apps or App Service.
- Azure Database for PostgreSQL.
- Azure Blob Storage.
- Azure Cache for Redis where needed.
- Azure Monitor or Application Insights.
- GitHub Actions or Azure DevOps.

An AWS equivalent may use:

- Elastic Container Registry.
- ECS or App Runner.
- RDS PostgreSQL.
- S3.
- ElastiCache.
- CloudWatch.

22.2 Deployment principles

- Keep the deployment simple.
 - Separate development and production configuration.
 - Use managed databases.
 - Use object storage for images.
 - Use HTTPS.
 - Run containers as non-root.
 - Store secrets outside images.
 - Back up the database.
 - Apply database migrations deliberately.
 - Avoid Kubernetes until there is a demonstrated need.
-

23. Project phases and acceptance gates

Phase 1: Repository and architecture

Deliver:

- Repository structure.
- Architecture diagram.
- Initial ADRs.
- Docker Compose skeleton.
- CI skeleton.
- Basic frontend and API health checks.

Acceptance gate:

- Frontend, API and database start locally.
- CI builds both frontend and backend.
- Architecture choices are documented.

Phase 2: Dataset and baseline model

Deliver:

- Dataset ingestion.
- Dataset report.
- Reproducible split.
- Baseline classifier.
- Evaluation notebook or report.
- MLflow experiment.

Acceptance gate:

- Baseline training can be reproduced.
- Metrics and artefacts appear in MLflow.
- A saved model can run inference on a sample image.

Phase 3: Core backend

Deliver:

- Authentication.
- Database schema.
- Inspection API.
- Image upload.
- Model metadata.
- Database migrations.

Acceptance gate:

- Authenticated user can create an inspection and upload an image.

- Data persists after restart.
- Backend tests pass.

Phase 4: Inference worker

Deliver:

- Redis queue.
- Worker process.
- PyTorch inference.
- Result persistence.
- Job-status API.
- Retry and failure handling.

Acceptance gate:

- Submitted jobs process asynchronously.
- Results include model version and latency.
- Duplicate retries do not create duplicate results.

Phase 5: Core frontend

Deliver:

- Login.
- Dashboard.
- Inspection creation.
- Upload interface.
- Job progress.
- Result display.

Acceptance gate:

- Complete workflow can be performed through the browser.
- Main pages have loading, empty and error states.
- Frontend tests pass.

Phase 6: Segmentation and review

Deliver:

- Segmentation model.
- Overlay generation.
- Image viewer.
- Review queue.
- Reviewer decisions.
- Audit records.

Acceptance gate:

- Reviewer can inspect a defect overlay and save a decision.

- Decision appears after page reload.
- Historical inference data remains immutable.

Phase 7: Model management

Deliver:

- Model list.
- Lifecycle states.
- Default-model selection.
- MLflow linkage.
- Model card.
- Model comparison.

Acceptance gate:

- Authorised user can activate a model.
- New jobs use the new default model.
- Old results retain their original model reference.

Phase 8: Hardening

Deliver:

- Security controls.
- Structured logging.
- Health endpoints.
- Full integration testing.
- Dependency scans.
- Container scans.
- Performance measurements.

Acceptance gate:

- CI is green.
- Security checklist is complete.
- System recovers from worker restart.
- Smoke tests pass from a clean environment.

Phase 9: Portfolio packaging

Deliver:

- Polished README.
- Architecture diagrams.
- Demo video.
- Screenshots.
- Model report.
- Threat model.
- ADRs.
- Public deployment where practical.

- Resume-ready project description.

Acceptance gate:

- A reviewer can understand, run and evaluate the system without verbal explanation.
-

24. Architecture decision records

At minimum, create ADRs for:

1. Modular monolith rather than microservices.
2. FastAPI as the backend framework.
3. React and TypeScript as the frontend.
4. PostgreSQL as the system of record.
5. Redis-backed asynchronous inference.
6. PyTorch as the training framework.
7. MLflow for experiment tracking.
8. Object storage rather than storing images in PostgreSQL.
9. Human approval before model activation.
10. Docker Compose for local orchestration.
11. ONNX as an optional deployment format.
12. Public industrial data only.

Each ADR should contain:

- Context.
 - Decision.
 - Alternatives considered.
 - Consequences.
 - Status.
-

25. Documentation deliverables

25.1 README

The README must include:

- Project summary.
- Screenshot or architecture image.
- Demonstrated skills.
- Core features.
- Stack.
- Quick start.
- Demo credentials.
- Test instructions.
- Training instructions.
- Repository structure.

- Known limitations.
- Future work.

25.2 Architecture documentation

Include:

- Context diagram.
- Container diagram.
- Component diagram.
- Inference sequence diagram.
- Review workflow.
- Deployment diagram.
- Data-flow diagram.

25.3 ML documentation

Include:

- Dataset report.
- Baseline report.
- Experiment comparison.
- Final model card.
- Threshold-selection explanation.
- Error analysis.
- Known failure modes.

25.4 Security documentation

Include:

- Threat model.
- Data classification.
- Trust boundaries.
- Authentication approach.
- File-upload controls.
- Secret-management approach.
- Residual risks.

25.5 API documentation

FastAPI should generate OpenAPI documentation, supplemented with:

- Authentication explanation.
 - Example requests.
 - Example errors.
 - Pagination conventions.
 - Versioning policy.
-

26. Demonstration scenario

The final demo should follow a fixed narrative.

1. Open the dashboard.
2. Show current active model and review statistics.
3. Create a new inspection.
4. Upload a normal image and a defective image.
5. Submit inference.
6. Show the asynchronous job state.
7. Open the normal result.
8. Open the defective result.
9. Adjust mask opacity and inspect the highlighted region.
10. Accept one prediction.
11. Amend another.
12. Open the review dashboard.
13. Show the persisted review.
14. Open the model page.
15. Compare two model versions.
16. Open MLflow and show the corresponding training run.
17. Show CI results.
18. Show the architecture diagram.
19. Run the test suite.
20. Explain a known limitation and the next engineering improvement.

This demonstrates both technical implementation and engineering judgement.

27. Success criteria

The project is complete when:

- A clean clone can be started using documented commands.
- A user can authenticate.
- A user can create an inspection.
- Images can be uploaded safely.
- Inference runs asynchronously.
- Results include confidence and localisation.
- Results are tied to an immutable model version.
- A reviewer can accept, reject or amend a result.
- Review history persists.
- MLflow contains reproducible experiments.
- A model can be exported and loaded for inference.
- Backend, frontend and integration tests pass.
- CI builds and scans the application.
- The application is containerised.
- Security controls are documented and tested.
- Architecture decisions are recorded.
- The README explains how the system works.

- A demonstration can be completed without editing code.
-

28. Stretch goals

28.1 Active learning

Rank samples for manual review based on:

- Low confidence.
- High uncertainty.
- Model disagreement.
- Rare component type.
- New visual distribution.

28.2 Dual-model comparison

Run two models on the same image and display:

- Prediction differences.
- Mask differences.
- Confidence differences.
- Latency differences.

28.3 Data drift monitoring

Track:

- Image brightness.
- Contrast.
- Resolution.
- Colour distribution.
- Embedding distribution.
- Prediction distribution.

Flag material deviations from the training set.

28.4 Video processing

Add:

- Video upload.
- OpenCV frame extraction.
- Sampling configuration.
- Frame-level inference.
- Defect timeline.
- Exported annotated video.

28.5 C++ inference component

Implement an optional C++ inference executable using:

- OpenCV.
- ONNX Runtime.
- CMake.
- GoogleTest.

This would demonstrate the ability to move a model from Python research into a more performance-oriented systems environment.

28.6 Cloud infrastructure as code

Add Terraform for:

- Container environment.
- Database.
- Object storage.
- Registry.
- Secrets.
- Monitoring.

28.7 Enhanced observability

Add:

- OpenTelemetry.
- Prometheus.
- Grafana.
- Distributed request tracing.
- Alerting rules.

29. Resume description

ForgeSight — AI-Assisted Visual Inspection Platform

Developed a production-style computer-vision platform for industrial defect detection and segmentation using PyTorch, OpenCV, FastAPI, React, TypeScript and PostgreSQL. Implemented asynchronous inference, human review workflows, immutable model-version tracking, MLflow experiment management, containerised deployment and automated CI testing. Added secure image handling, role-based access control, audit logging, ONNX model export and end-to-end browser tests.

30. Interview talking points

The project should prepare the developer to discuss:

- Why a modular monolith was selected.
 - How training and inference preprocessing were kept consistent.
 - How model versions were made immutable.
 - How false negatives influenced threshold selection.
 - Why image data was stored outside PostgreSQL.
 - How inference jobs were made idempotent.
 - How human review data could support retraining.
 - How data leakage was prevented.
 - How model performance differs from operational performance.
 - How file-upload attacks were mitigated.
 - How CI differs for normal software and ML training.
 - Why Kubernetes was deliberately excluded.
 - How the system could be scaled.
 - How ONNX changes deployment options.
 - What the model cannot reliably detect.
 - What evidence would be required before industrial use.
-

31. Recommended minimum polished release

The minimum version worth presenting publicly should contain:

- One trained classifier.
- One segmentation or anomaly-localisation model.
- MLflow experiment history.
- FastAPI backend.
- PostgreSQL persistence.
- Redis-backed worker.
- React and TypeScript frontend.
- Image overlay viewer.
- Human review queue.
- Model-version page.
- Docker Compose deployment.
- Backend and frontend tests.
- One complete Playwright workflow.
- CI pipeline.
- Threat model.
- Architecture diagram.
- Model card.
- Demo video.
- Clear README.

Anything below this risks appearing as another model demonstration rather than an engineered AI system.